

Numerical-Parity Transfer of a Compact Load-Forecasting and Residual-Decision Model to ESP8266

Abstract—Embedded machine learning models deployed on resource-constrained microcontrollers can diverge from their high-level reference implementations after feature reordering, numeric conversion, or device integration, potentially undermining the reliability of edge forecasting and decision systems. Although compact forecasting and residual-decision models are increasingly considered for low-resource devices such as the ESP8266, complete row-level numerical-parity verification across the reference-to-device pipeline with whole-firmware resource measurement remains unexplored. Therefore, this research aims to conduct a reproducibility and deployment audit of a compact load-forecasting and residual-decision computation from a high-level reference implementation to physical ESP8266 execution. To achieve these objectives, firstly, six forecasting approaches were evaluated on 5,000 half-hourly observations from a public normalized smart-meter-style dataset using chronological holdout and expanding-window validation. Secondly, five residual- and isolation-based anomaly detection methods were compared using publisher-supplied labels as secondary context. Finally, the nine-parameter standardized linear model was exported as C++ constants and validated across the host reference, independently compiled C++, and physical ESP8266EX implementations. The model reproduced all 933 held-out predictions and decisions with a maximum device difference of 8.9×10^{-8} normalized units and achieved an average kernel time of 50.427 μ s per prediction. Whole-firmware instruction RAM utilization reached 92%. As such, this research contributes a transparent row-level reproducibility and deployment-audit methodology for compact edge-forecasting models.

Index Terms—chronological load forecasting, ESP8266, TinyML, hardware-in-the-loop, numerical parity, residual thresholding, reproducibility

I. INTRODUCTION

Short-horizon load forecasting is commonly evaluated with recurrent networks, tree ensembles, or statistical baselines on household and building data [1], [2], [3]. Prediction residuals and unsupervised models have separately been used to flag unusual consumption [4], [5], [6], and edge-oriented work has examined cloud-edge collaboration, federated split learning, and embedded energy analytics [7], [8], [9]. This combination is established prior art, and general TinyML benchmarks such as MLPerf Tiny and MCUNet report how an isolated model kernel behaves once optimized for a target architecture [10], [11].

What these efforts do not report is whether one specific trained model, written first in Python, survives feature reordering, independent re-implementation, and physical replay on a named low-resource board with its full application firmware attached, rather than an isolated kernel. A model that appears

adequate offline can still silently change its predictions or threshold decisions after such a transfer, and kernel-only benchmarks are not designed to expose this; papers reporting a single-board deployment typically stop at “it runs on the device” rather than quantifying row-level numerical agreement against the reference computation. This reproducibility gap is the one this paper targets.

Therefore, the objectives of this research are to conduct a complete, row-level reproducibility and deployment audit of a compact forecasting and residual-decision computation across three independent execution layers, and to report the embedded resource cost of the firmware carrying it. The ESP8266EX was selected deliberately over a more contemporary board (ESP32, Cortex-M4): as a low-cost, widely deployed legacy target with tight, separately budgeted instruction RAM, it makes transfer failures and integration bottlenecks visible in ways a more generous platform would mask; a like-for-like ESP32 comparison is deferred to Section VII. To attain this objective, this paper makes the following contributions:

- A leakage-aware, HAC/Holm-adjusted chronological comparison of six forecasters, including the compact edge model, with threshold-sensitivity behavior of the residual decision rule reported against publisher-supplied labels as secondary context only.
- A complete row-level, three-layer evidence chain – Python reference, independently compiled C++, and physical ESP8266 replay – covering every one of 933 held-out vectors rather than a spot check, with a quantified sub- 10^{-4} parity tolerance.
- Whole-firmware, not isolated-kernel, timing, RAM, instruction-RAM (IRAM), and flash measurements, surfacing an instruction-RAM bottleneck that kernel-only TinyML benchmarks do not expose.

No claim of forecasting-algorithm novelty, calibrated smart-meter operation, or verified anomaly detection is made; the paper is positioned throughout as a reproducibility and deployment-audit case study.

The remainder of this paper is organized as follows. Section II reviews related work and situates the reproducibility gap. Section III describes the dataset, evaluation protocol, secondary anomaly-agreement analysis, and hardware-in-the-loop procedure. Section IV presents forecasting, anomaly-agreement, and parity results. Section V discusses claim boundaries, Section VI states limitations, and the remaining

sections cover reserved and future work before concluding.

II. RELATED WORK AND RESEARCH GAP

Deep recurrent models have improved household forecasting on larger, physically grounded datasets [1], [2], [3]. Prediction-based anomaly detection has also been evaluated online and in multitier edge architectures [4], [5], and Hernandez et al. studied activity-related anomalies using data from a commercial meter, providing event context absent from the present dataset [6].

For embedded intelligence, Hu and Tang presented an IoT smart-metering edge architecture [7]; Li et al. demonstrated federated split learning across Cortex-M4 meters and edge computers [8]; and Gajendran et al. used ESP8266 nodes for data acquisition while evaluating forecasting on computer hardware rather than the device itself [12]. Kolosov et al. combined calibrated high-frequency sensing with embedded energy disaggregation on substantially stronger platforms [9]. MLPerf Tiny and MCUNet standardize how an isolated kernel is measured or optimized across architectures [10], [11], but neither reports whether a specific trained model, and its threshold decision, survives re-implementation and physical replay with its application firmware attached; a model passing a kernel-level benchmark could still fail this row-level transfer audit, and vice versa.

In sum, existing work highlights several gaps. First, embedded forecasting papers commonly show that a model runs on a device without quantifying row-level numerical agreement against the reference implementation, so transfer correctness is asserted rather than measured. Second, general TinyML benchmarks characterize an isolated kernel rather than a model’s behavior inside its whole application firmware, leaving integration-level resource constraints underreported. Third, embedded case studies rarely separate forecasting-quality, anomaly-label-agreement, and deployment-fidelity evidence, which can make a modest algorithmic contribution appear more validated than it is. This study addresses the first two gaps with the three-layer Python/C++/ESP8266 parity protocol and whole-firmware measurements in Sections III–IV, and the third gap by explicitly separating forecasting, secondary anomaly-agreement, and deployment evidence throughout, consistent with its framing as a reproducibility and deployment-audit study rather than a forecasting- or anomaly-detection contribution.

III. MATERIALS AND METHODS

A. Dataset identity and scope

The analyzed CSV is version 1 of the Kaggle listing *Smart Meter Electricity Consumption Dataset* [13]. Its SHA-256 fingerprint is D48DF940FD29444F...D9D5F9DC88F588. The table contains 5,000 rows and seven columns from 1 January through 14 April 2024 at 30-minute intervals. After constructing a seven-day lag, 4,664 rows remain. No missing cells, duplicate timestamps, or non-30-minute gaps were found.

The continuous columns lie approximately in $[0, 1]$, but the listing does not provide inverse-scaling metadata or a primary measurement protocol. Accordingly, MAE, RMSE, thresholds,

and numeric differences are reported only in normalized target units. The 5% “Abnormal” labels are publisher-described algorithmic labels, not independently verified electrical events. They are used for method agreement comparisons rather than physical fault claims.

B. Features and chronological evaluation

The common predictor set comprises target lags 1, 2, 48, and 336 plus hour, day of week, month, and weekend indicator. Each lag references an observation available before its prediction timestamp. Six approaches were compared: lag-1 persistence, lag-48 seasonal naive, ordinary linear regression, random forest (200 trees, maximum depth 12, minimum leaf size 2), histogram gradient boosting, and a standardized linear edge model. Random seeds were fixed at 42 where applicable.

The single holdout uses 3,731 earlier rows for fitting and 933 later rows for testing. For the edge model, the earlier segment is divided into 2,984 fitting rows and 747 threshold-calibration rows. The expanding-window protocol contains five folds, each with 400 calibration and 400 test rows, yielding 2,000 non-overlapping test predictions; MAE, RMSE, and sMAPE are reported. Fold-level 95% Student-*t* intervals are descriptive given only five folds.

Paired absolute-error differences use a moving-block percentile bootstrap with 10,000 replicates and block length 48. A Diebold–Mariano-style mean loss-differential statistic uses a Bartlett Newey–West HAC variance with lag 48; five comparisons are Holm-adjusted. Statistical significance is not treated as evidence of cross-dataset generalization.

C. Residual and isolation-based decisions (secondary, contextual evidence)

This subsection is reported as secondary context for the device decision rule, not as a validated anomaly-detection contribution: agreement is measured only against publisher-supplied algorithmic labels rather than independently verified electrical events. The device rule compares absolute forecast residuals with the 99th percentile of 747 chronological calibration residuals, giving a threshold of 0.383026 normalized units, with sensitivity evaluated at percentiles 90, 92.5, 95, 97.5, 99, and 99.5. Additional methods were a rolling-week p99 threshold, adaptive EWMA p99 threshold, split-conformal 95% threshold, and Isolation Forest with 5% contamination, described by precision, recall, F1, confusion counts, and alert rate against the supplied labels. The retrospectively best threshold is not proposed as a deployable operating point, since choosing it on final test labels would bias evaluation.

D. Model export and hardware-in-the-loop protocol

The selected device computation is standardized linear regression with eight coefficients and one intercept. Together with eight means, eight scales, and one residual threshold, its core numeric state contains 26 FP32 values (104 bytes), and each standardized prediction contains eight subtractions, divisions, multiplications, and accumulation additions at source level. The generated header fixes feature order and constants;

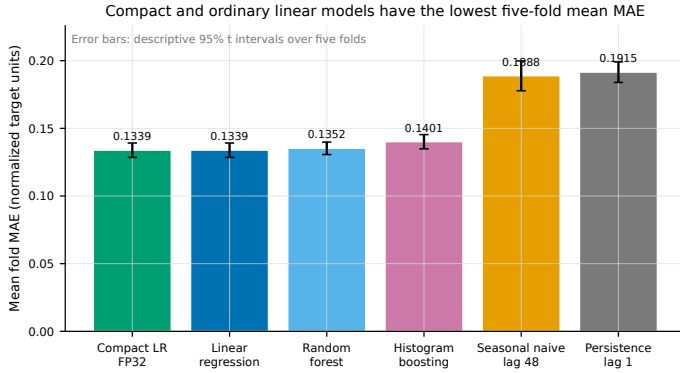


Fig. 1. Forecasting comparison using normalized target units. Error bars summarize fold-level variation; they do not establish performance on other datasets.

Random Forest and Isolation Forest are not claimed to run unchanged on the device.

Three evidence layers were kept separate: Python float64 reference evaluation, an independently compiled C++17 FP32 host executable, and physical ESP8266EX NodeMCU 1.0 replay. In the full hardware-in-the-loop (HIL) test, all 933 held-out vectors, Python predictions, and decisions were stored in program flash, and the board recomputed predictions, residual decisions, forecasting metrics, and confusion counts. This replay validates arithmetic and integration but not live lag-buffer operation or sensor acquisition.

Pure model-kernel time was recorded as 30 batch averages, each over 2,000 predictions. Compiler summaries supplied whole-firmware RAM, instruction-RAM (IRAM), and flash-code use for normal and full-validation builds. These totals include the ESP8266 core, Wi-Fi, HTTP, serial reporting, and application code; they are not isolated model footprints. ThingSpeak HTTP responses demonstrate prototype telemetry continuity only. No calibrated sensing interface or mains connection was used.

IV. RESULTS

A. Forecasting performance

Table I summarizes the expanding-window experiment. The compact FP32 and ordinary linear models are numerically indistinguishable at the reported precision, since they implement the same affine mapping with different serialization, and both outperform naive baselines on this dataset. The compact model's 1.02% MAE difference relative to random forest is statistically distinguishable under the stated test, but the absolute difference is small and should not be generalized beyond this case study.

Feature ablation produced mean MAE values from 0.133823 to 0.133890 across lag, calendar, and weather groupings; these small differences do not justify a claim that weather features drive forecasting performance. The eight-feature lag/calendar set was retained for deployment, since it avoids external weather inputs while matching the tested alternatives closely.

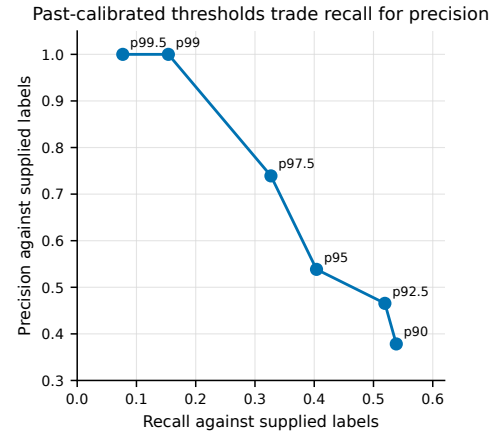


Fig. 2. Secondary, de-emphasized anomaly-agreement context: precision–recall relative to the supplied labels, which are not independent physical anomaly ground truth.

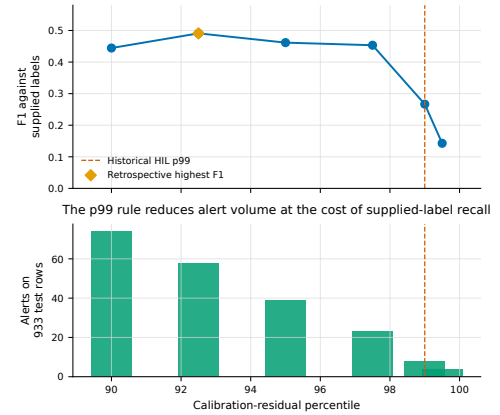


Fig. 3. Static-threshold sensitivity (secondary evidence). Selecting a threshold from these test results would bias a final evaluation.

B. Anomaly-method and threshold sensitivity (secondary, de-emphasized evidence)

These results describe agreement with algorithm-generated labels, not verified fault detection. They are reported only as secondary context for the device threshold rule, not as an anomaly-detection contribution, and should be weighted well below the forecasting and cross-platform parity results reported elsewhere in this section. Table III shows that supplied-label agreement depends strongly on method and alert volume: split conformal produced the highest observed F1 (0.462), while the deployed p99 rule generated no false positives but missed 44 of 52 supplied positives. At the retrospectively best static percentile (92.5), F1 reached 0.491 with 58 alerts; this single descriptive figure was not used to select the final device rule and is not proposed as an operating point.

C. Cross-platform parity and embedded measurements

The host C++ executable matched all 933 predictions and decisions, with maximum absolute difference 7.557×10^{-8} normalized units. The physical ESP8266 also matched all 933 predictions and decisions; its maximum difference was

TABLE I
FIVE-FOLD EXPANDING-WINDOW FORECASTING RESULTS. MAE AND RMSE USE NORMALIZED TARGET UNITS.

Model	MAE mean $\pm$ SD	MAE 95% CI	RMSE mean $\pm$ SD	Pooled sMAPE (%)
Compact edge linear (FP32)	0.133858 $\pm$ 0.004271	[0.128555, 0.139161]	0.165970 $\pm$ 0.004175	40.354
Linear regression	0.133858 $\pm$ 0.004271	[0.128555, 0.139161]	0.165970 $\pm$ 0.004175	40.354
Random forest	0.135236 $\pm$ 0.003713	[0.130626, 0.139846]	0.167993 $\pm$ 0.003135	40.593
Histogram gradient boosting	0.140116 $\pm$ 0.004235	[0.134857, 0.145375]	0.174222 $\pm$ 0.003543	41.882
Seasonal naive (lag 48)	0.188809 $\pm$ 0.008865	[0.177802, 0.199816]	0.235346 $\pm$ 0.008103	57.707
Persistence (lag 1)	0.191500 $\pm$ 0.006112	[0.183910, 0.199089]	0.237066 $\pm$ 0.007700	58.497

TABLE II
PAIRED MAE COMPARISONS AGAINST THE COMPACT FP32 MODEL OVER 2,000 PREDICTIONS.

Comparator	Difference	Bootstrap 95% CI	p_{Holm}
Linear regression	-6.82×10^{-11}	$[-4.44, 3.78] \times 10^{-10}$	0.745
Random forest	-0.001378	$[-0.002490, -0.000180]$	0.0345
Hist. gradient boost	-0.006258	$[-0.008475, -0.003911]$	1.92×10^{-7}
Seasonal naive	-0.054951	$[-0.059999, -0.050036]$	2.27×10^{-97}
Persistence	-0.057642	$[-0.062364, -0.052358]$	1.77×10^{-114}

TABLE III
AGREEMENT WITH PUBLISHER-SUPPLIED ALGORITHMIC LABELS ON THE 933-ROW TEST SEGMENT (SECONDARY, DE-EMPHASIZED EVIDENCE).

Method	TP	FP	FN	TN	Precision	Recall	F1	Alerts (%)
Split conformal (95%)	21	18	31	863	0.538	0.404	0.462	4.18
Adaptive EWMA p99	16	14	36	867	0.533	0.308	0.390	3.22
Rolling-week p99	10	1	42	880	0.909	0.192	0.317	1.18
Static residual p99	8	0	44	881	1.000	0.154	0.267	0.86
Isolation Forest (5%)	19	114	33	767	0.143	0.365	0.205	14.26

8.9×10^{-8} . Device-derived MAE, RMSE, and confusion counts matched the Python record within the declared tolerance. These results support numerical portability, not detector adequacy.

Across 30 batches, model-kernel timing had mean $50.427 \mu\text{s}$, sample standard deviation $0.072 \mu\text{s}$, median $50.403 \mu\text{s}$, 95th percentile $50.539 \mu\text{s}$, and range 50.362 – $50.700 \mu\text{s}$ per prediction. They are batch averages for one firmware/compiler/board setup, not individual-cycle, sensing-to-alert, or network-latency measurements.

Offline representation tests found identical decisions for standardized FP32, fused affine FP32, Q7/Q30, and Q15/Q30 forms over all 933 rows. Q15/Q30 reduced core coefficient/bias state from 104 bytes to 40 bytes (plus generic input-range metadata) with maximum deviation 1.211×10^{-5} normalized units. Because physical fixed-point timing and whole-firmware measurements were not completed, the FP32 implementation remains the measured device result.

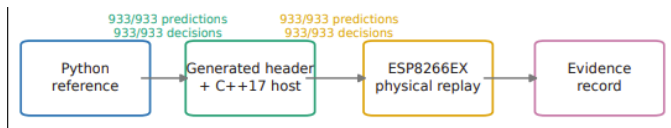


Fig. 4. Evidence layers for Python, host C++, and ESP8266 execution. Complete parity applies to fixed held-out-vector replay.

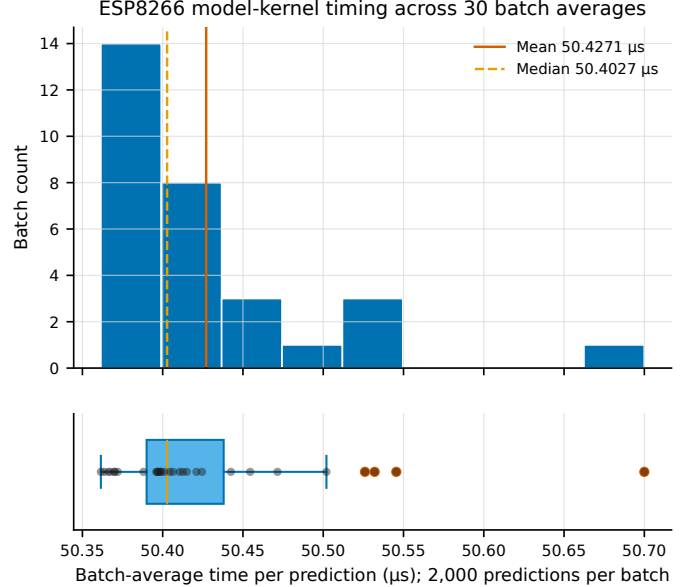


Fig. 5. Measured embedded costs: distribution of 30 ESP8266 batch-average model-kernel measurements, each covering 2,000 predictions.

V. DISCUSSION

The central result is the separation of forecasting evidence, label-agreement evidence, and deployment evidence. Chronological tests show that a compact affine model is competitive

TABLE IV
PYTHON-TO-DEVICE PARITY AND WHOLE-FIRMWARE RESOURCE EVIDENCE.

Measurement	Observed result	Claim boundary
Host C++ prediction/decision agreement	933/933; 933/933	Generated-header arithmetic, not physical execution
ESP8266 prediction/decision agreement	933/933; 933/933	Flash-resident held-out replay, not live sensing
Maximum device difference	8.9×10^{-8} normalized units	Below the 10^{-4} parity tolerance
Kernel timing	$50.427 \mu\text{s}$ mean; $30 \times 2,000$	Excludes sensing and communication
Normal firmware RAM / IRAM / flash	29,516 / 60,343 / 257,796 bytes	36.81% / 92.08% / 24.59%
Validation firmware RAM / IRAM / flash	29,852 / 60,343 / 298,132 bytes	37.23% / 92.08% / 28.43%

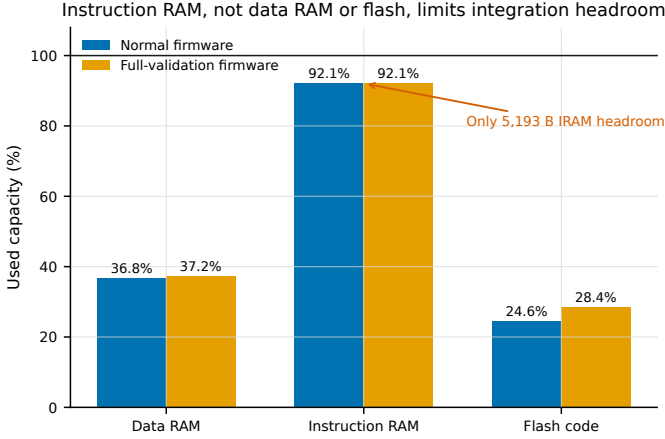


Fig. 6. Measured embedded costs: whole-firmware capacity use, showing IRAM as the principal measured integration constraint.

with the evaluated alternatives on this normalized dataset, but the similarity of feature-ablation results and the short 104-day span limit interpretation. The paired comparison against random forest is statistically distinguishable under the declared procedure, yet its absolute MAE difference is only 0.001378 normalized units, so simplicity is justified here by portability and auditability rather than a broad accuracy claim.

The anomaly results are secondary, weaker, and de-emphasized by design: the static device threshold gives perfect precision only because it issues eight alerts and misses most supplied positives, and the supplied labels are not independent physical events. Neither this precision value nor cross-platform decision agreement establishes reliable fault detection; threshold sensitivity instead demonstrates an operational trade-off to be revisited with verified events, and this section should not be read as the paper’s main result.

Complete cross-platform parity is meaningful for implementation assurance: it rules out several common transfer failures over the tested holdout and confirms the complete computation runs on the physical board. It does not show that the ESP8266 has received a calibrated load signal, maintained the 336-sample live history, or generated an operational forecast. The 92% IRAM result also warns that adding TLS, signal processing, or sensing libraries may exceed this build’s remaining instruction memory even though flash and general RAM have more headroom.

VI. LIMITATIONS

The dataset has undocumented primary provenance, normalized values without inverse scaling, a short time span, and algorithm-generated anomaly labels; only one dataset and one board were evaluated, with a prespecified one-day block/HAC lag over five folds. The HIL experiment replays stored features rather than a live lag buffer. Timing is a kernel batch average; no end-to-end latency, current, power, or energy-per-inference measurement was made. Prototype telemetry uses HTTP and is unsuitable for protecting field credentials, and no calibrated voltage/current/power/energy sensor was evaluated. These limitations prohibit claims of physical kWh accuracy, secure operation, energy efficiency, real-world anomaly performance, or general deployment readiness.

VII. RESERVED EXTENSION AND FUTURE WORK

An ESP32 field-validation extension is intentionally reserved for future reporting: it will add the sensor and conditioning circuit, calibration protocol, independently recorded reference measurements, field-data summary, event-labeling method, and an ESP8266/ESP32 comparison once those experiments are complete, with no field result claimed here. The evaluation pipeline, header generator, and HIL harness are dataset- and board-agnostic and are meant to be reused; priorities are an openly documented, multi-season meter dataset with verified events; an isolated, calibrated sensing stage tested for startup, missing observations, clock drift, and restart recovery; thresholds prespecified on calibration data and evaluated once on independent events; measured end-to-end latency, power, and energy per inference; migration from HTTP to HTTPS or authenticated MQTT; and the same parity, timing, and resource protocol applied to the Q15/Q30 candidate before any replacement decision.

VIII. CONCLUSION

This study demonstrates a reproducible method for transferring a compact forecasting and residual-decision computation from Python to a physical ESP8266. Across the complete 933-row held-out replay, host C++ and device execution preserved every prediction match and every threshold decision within sub- 10^{-7} normalized-unit difference, with the board averaging $50.427 \mu\text{s}$ per prediction over 30 batches and whole-firmware IRAM use reaching 92%. Forecasting and anomaly comparisons provide context but do not convert the normalized

dataset or publisher-supplied labels into calibrated physical evidence: the defensible outcome is implementation fidelity with explicit resource and validity boundaries, not a new forecasting algorithm or a deployable smart meter.

DATA AND CODE AVAILABILITY

The source code, model parameters, firmware, experimental results, data split indices, figures, and reproduction instructions have been prepared in a private repository. To preserve the anonymity of the review process, the repository is not publicly linked in this submission. The repository will be made publicly accessible upon camera-ready acceptance. The raw dataset and row-level validation vectors are not redistributed. To support reproducibility, the dataset source URL, version, SHA-256 fingerprint, and data regeneration procedure are documented. Private Wi-Fi and telemetry credentials are excluded from the repository. The author acknowledges the dataset maintainers and the open-source Python, Arduino, ESP8266, and scientific-computing tools used in this work.

REFERENCES

- [1] H. Shi, M. Xu, and R. Li, "Deep Learning for Household Load Forecasting—A Novel Pooling Deep RNN," *IEEE Transactions on Smart Grid*, vol. 9, no. 5, pp. 5271–5280, 2018. [Online]. Available: <https://doi.org/10.1109/TSG.2017.2686012>
- [2] W. Kong, Z. Y. Dong, Y. Jia, D. J. Hill, Y. Xu, and Y. Zhang, "Short-Term Residential Load Forecasting Based on LSTM Recurrent Neural Network," *IEEE Transactions on Smart Grid*, vol. 10, no. 1, pp. 841–851, 2019. [Online]. Available: <https://doi.org/10.1109/TSG.2017.2753802>
- [3] K. Aurangzeb, S. I. Haider, and M. Alhussein, "Individual Household Load Forecasting Using Bi-Directional LSTM Network with Time-Based Embedding," *Energy Reports*, vol. 11, pp. 3963–3975, 2024. [Online]. Available: <https://doi.org/10.1016/j.egy.2024.03.028>
- [4] X. Liu and P. S. Nielsen, "Scalable Prediction-Based Online Anomaly Detection for Smart Meter Data," *Information Systems*, vol. 77, pp. 34–47, 2018. [Online]. Available: <https://doi.org/10.1016/j.is.2018.05.007>
- [5] D. Utomo and P.-A. Hsiung, "A Multitiered Solution for Anomaly Detection in Edge Computing for Smart Meters," *Sensors*, vol. 20, no. 18, p. 5159, 2020. [Online]. Available: <https://doi.org/10.3390/s20185159>
- [6] A. Hernández, R. Nieto, L. de Diego-Otón, M. C. Pérez-Rubio, J. M. Villadangos-Carrizo, D. Pizarro, and J. Ureña, "Detection of Anomalies in Daily Activities Using Data from Smart Meters," *Sensors*, vol. 24, no. 2, p. 515, 2024. [Online]. Available: <https://doi.org/10.3390/s24020515>
- [7] H. Hu and L. Tang, "Edge Intelligence for Real-Time Data Analytics in an IoT-Based Smart Metering System," *IEEE Network*, vol. 34, no. 5, pp. 68–74, 2020. [Online]. Available: <https://doi.org/10.1109/MNET.011.2000039>
- [8] Y. Li, D. Qin, H. V. Poor, and Y. Wang, "Introducing Edge Intelligence to Smart Meters via Federated Split Learning," *Nature Communications*, vol. 15, no. 1, p. 9044, 2024. [Online]. Available: <https://doi.org/10.1038/s41467-024-53352-9>
- [9] D. Kolosov, M. Robinson, P. A. Schirmer, and I. Mporas, "An Instrumental High-Frequency Smart Meter with Embedded Energy Disaggregation," *Sensors*, vol. 25, no. 17, p. 5280, 2025. [Online]. Available: <https://doi.org/10.3390/s25175280>
- [10] C. Banbury, V. J. Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kiraly, P. Montino, D. Kanter, S. Ahmed, D. Pau, U. Thakker, A. Torrini, P. Warden, J. Cordaro, G. D. Guglielmo, J. Duarte, S. Gibellini, V. Parekh, H. Tran, N. Tran, N. Wenxu, and X. Xuesong, "MLPerf Tiny Benchmark," in *NeurIPS Datasets and Benchmarks Track*, 2021. [Online]. Available: <https://openreview.net/forum?id=8RxxwAut1BI>
- [11] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "MCUNet: Tiny Deep Learning on IoT Devices," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 11 711–11 722. [Online]. Available: <https://papers.nips.cc/paper/2020/hash/86c51678350f656dcc7f490a43946ee5-Abstract.html>
- [12] G. P. R. V. and D. S. N., "IoT-Based Home Energy Management Using Machine Learning and BiLSTM-GRU Forecasting," *AIP Advances*, vol. 16, no. 7, p. 075012, 2026. [Online]. Available: <https://doi.org/10.1063/5.0339511>
- [13] ziya07, "Smart Meter Electricity Consumption Dataset," Kaggle dataset, version 1, accessed 2026-08-26; listing displays CC0. [Online]. Available: <https://www.kaggle.com/datasets/ziya07/smart-meter-electricity-consumption-dataset>